

Kapitel 3: Grundlegende Container

Arrays & dynamische Arrays | Verkettete Listen | Stack, Queue, Deque

1 Arrays (Felder)

Definition 3.1 — Statisches Array

Ein **Array** a der Größe n speichert n Elemente an aufeinanderfolgenden Speicheradressen.

Direkte Adressierung (homogene Einträge, Elementgröße s Bytes):

$$\text{addr}(a[i]) = \text{addr}(a[0]) + i \cdot s$$

Bei **heterogenen Einträgen** (z.B. Python-list): Referenzen/Pointer gespeichert, Zugriff trotzdem $\mathcal{O}(1)$.

Operation	Komplexität	Grund
$a[i]$ lesen / schreiben	$\mathcal{O}(1)$	direkte Adressierung
$\text{len}(a)$	$\mathcal{O}(1)$	gespeicherter Zähler
$\text{append}(v)$ naiv	$\mathcal{O}(n)$	gesamtes Array kopieren
insert in Mitte	$\mathcal{O}(n)$	alle Folgeelemente verschieben

2 Unbeschränkte (dynamische) Arrays

Definition 3.2 — Dynamisches Array mit Verdoppelungsstrategie

Ein dynamisches Array hält **Kapazität** $w \geq n$ (tatsächliche Größe) vor.

Verdoppelungsstrategie mit Wachstumsfaktor L und Schrumpffaktor B ($L < B$ erforderlich!):

- **append**: falls $n = w$ (*voll*) $\rightarrow \text{reallocate}$ auf $w' = L \cdot w$ (typisch $L = 2$)
- **pop**: falls $n < w/B$ (*zu leer*) $\rightarrow \text{reallocate}$ auf $w' = w/B$ (typisch $B = 4$)

Warum $L < B$? Nach einem Schrumpfen (neue Kapazität w/B) muss $n/w < 1/L$ gelten, damit nicht sofort wieder gewachsen wird. Mit $L = 2, B = 4$: nach Schrumpfen auf $w/4$ liegt $n < w/4$, Wachstum erst ab $n = w/4 = w'/2$ — kein direktes Hin- und Herkopieren.

Satz 3.3 — Amortisierte $\mathcal{O}(1)$ pro append/pop

Sei $L = 2, B = 4$. Jede Folge von m **append**- und **pop**-Operationen kostet insgesamt $\mathcal{O}(m)$ Schritte. Pro Operation also amortisiert $\mathcal{O}(1)$.

3 Amortisierte Analyse

Definition 3.4 — Framework der amortisierten Analyse

- Z = Menge der Operationstypen; S = Zustand der Datenstruktur (size, capacity, ...)
- $T_x(s)$ = tatsächliche Kosten von Operation x im Zustand s
- $A_x(s)$ = amortisierte Kosten (Schätzung)

Korrektheitsbedingung: Für jede Operationsfolge $o_1, \dots, o_m$ muss gelten:

$$\sum_{k=1}^m T_{o_k}(s_{k-1}) \leq \sum_{k=1}^m A_{o_k}(s_{k-1})$$

d. h. die tatsächlichen Gesamtkosten werden durch die amortisierten *nie unterschätzt*.

Definition 3.5 — Bankkonto-Methode (Accounting Method)

Jede Operation zahlt **Credits** ein oder hebt ab. Der **Kontostand** $\Phi \geq 0$ muss stets nicht-negativ bleiben.

Für dynamisches Array ($L = 2$, $B = 4$):

Operation	Tatsächl. Kosten	Credits eingezahlt / abgehoben
append (kein reallocate)	1	+2 Credits einzahlen
pop (kein reallocate)	1	+1 Credit einzahlen
reallocate (Größe n)	n	$-n$ Credits abheben

Beweis-Idee (append-Richtung): Nach **reallocate** auf $w = 2n$ enthält das Array n Elemente. Seit dem letzten **reallocate** wurden mindestens n **append**-Operationen ausgeführt — jede hat 2 Credits eingezahlt, also stehen mindestens n Credits bereit, die die n Kopierschritte des nächsten **reallocate** decken. $\square$

Beispiel 3.6 — Binärzähler-Inkrementierung (amortisiert $\mathcal{O}(1)$)

Problem: m Inkrementierungen eines k -Bit-Binärzählers. Kosten = Anzahl geflippter Bits.

Analyse: Bit j flippt genau alle 2^j Schritte:

$$\text{Gesamtkosten} = \sum_{j=0}^{\lfloor \log_2 m \rfloor} \left\lfloor \frac{m}{2^j} \right\rfloor \leq \sum_{j=0}^{\infty} \frac{m}{2^j} = 2m \in \mathcal{O}(m)$$

$\Rightarrow$ amortisiert $\mathcal{O}(1)$ pro Inkrementierung.

Bankkonto-Argument: Jede Inkrementierung zahlt 2 Credits ein (1 für aktuellen Flip, 1 gespart). Das Zurücksetzen von k Bits kostet k Credits — genau die zuvor eingezahlten.

Achtung

Prüfungsformat: Bei der Bankkonto-Methode muss man (1) die Kreditzuweisung explizit angeben, (2) zeigen, dass der Kontostand niemals negativ wird, und (3) daraus die amortisierten Gesamtkosten ableiten.

4 Doppelt verkettete Liste

Definition 3.7 — Doppelt verkettete Liste mit Dummy-Knoten

Jeder **Knoten** enthält: **value**, **next** (Zeiger auf Nachfolger), **prev** (Zeiger auf Vorgänger).

Dummy-/Sentinel-Knoten: Ein spezieller Knoten ohne Datenwert bildet Anfang und Ende der Liste (oft als Ring). Vorteile:

- Kein Sonderfall für leere Liste oder erstes/letztes Element.
- Code vereinfacht sich deutlich — alle Operationen uniform.

Definition 3.8 — $\text{splice}(a, b, t)$: die fundamentale Operation

Semantik: Verschiebe die Teilfolge $\langle a, \dots, b \rangle$ (von Knoten a bis b) aus ihrer aktuellen Position und hänge sie *direkt hinter* Knoten t ein.

Vorbedingung: t liegt *nicht* zwischen a und b .

Komplexität: $\mathcal{O}(1)$ — nur sechs Zeiger werden umgehängt, unabhängig von Listenlänge.

Alle Standardoperationen reduzieren sich auf splice:

Operation	via splice	Komplexität
$\text{insert}(a, x)$	$\text{splice}(x, x, a)$	$\mathcal{O}(1)$
$\text{remove}(a)$	$\text{splice}(a, a, \text{dummy})$	$\mathcal{O}(1)$
$\text{moveAfter}(a, t)$	$\text{splice}(a, a, t)$	$\mathcal{O}(1)$
$\text{search}(x)$	linearer Durchlauf	$\mathcal{O}(n)$

Beispiel 3.9 — splice visuell

Ausgangslage: $\dots \rightarrow p \rightarrow [a \rightarrow \dots \rightarrow b] \rightarrow q \rightarrow \dots \quad \dots \rightarrow t \rightarrow u \rightarrow \dots$

Nach $\text{splice}(a, b, t)$: $\dots \rightarrow p \rightarrow q \rightarrow \dots \quad \dots \rightarrow t \rightarrow [a \rightarrow \dots \rightarrow b] \rightarrow u \rightarrow \dots$

Geänderte Zeiger: $p.\text{next} = q$, $q.\text{prev} = p$, $t.\text{next} = a$, $a.\text{prev} = t$, $b.\text{next} = u$, $u.\text{prev} = b$ — genau 6 Zuweisungen, also $\mathcal{O}(1)$.

Definition 3.10 — Speicherverwaltung: Freelist

Einzelne Knotenallokierungen (**new/malloc**) sind teuer (Systemaufruf). Besser:

- Alloziere einen Block für $k \geq 1$ Knoten auf einmal (z. B. als Array).
- **Freelist:** interne verkettete Liste der aktuell *freien* Knoten.
- **insert:** Knoten aus Freelist entnehmen $\rightarrow \mathcal{O}(1)$.
- **remove:** Knoten zurück in Freelist geben $\rightarrow \mathcal{O}(1)$.

5 Einfach verkettete Liste

Definition 3.11 — Einfach verkettete Liste

Jeder Knoten enthält nur **value** und **next** (kein **prev**).

- **Vorteil:** weniger Speicherbedarf pro Knoten.
- **Nachteil:** $\text{remove}(a)$ in $\mathcal{O}(1)$ nicht möglich (Vorgänger unbekannt).
- Stattdessen: $\text{removeAfter}(a)$ — entfernt den Nachfolger von a in $\mathcal{O}(1)$.

6 Vergleich: Array vs. Liste

Operation	Statisches Array	Dyn. Array	Doppelt verk. Liste
<code>a[i]</code> (wahlfreier Zugriff)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>insert</code> am Ende	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$	$\mathcal{O}(1)$
<code>insert</code> in der Mitte	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (mit Zeiger)
<code>remove</code> am Ende	$\mathcal{O}(1)$	$\mathcal{O}(1)^*$	$\mathcal{O}(1)$
<code>remove</code> in der Mitte	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (mit Zeiger)
<code>size()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$ oder $\mathcal{O}(n)$
Speicherlayout	kompakt	kompakt	fragmentiert

* amortisiert

Achtung

Keine universell beste Implementierung! Die Wahl hängt vom Zugriffsmuster ab: häufiger wahlfreier Zugriff → Array; häufiges Einfügen/Löschen in der Mitte mit bekanntem Zeiger → Liste.

7 Abstrakte Container: Stack, Queue, Deque

Definition 3.12 — Stack, Queue, Deque

Stack (Stapel) — **LIFO** (*Last In, First Out*):

- Operationen: `push(v)`, `pop()`, `top()`
- Implementierung: dynamisches Array (`append/pop`) oder einfach verk. Liste
- Alle Operationen $\mathcal{O}(1)$ (amortisiert beim Array)

Queue (Warteschlange) — **FIFO** (*First In, First Out*):

- Operationen: `pushBack(v)`, `popFront()`
- Implementierung: doppelt verkettete Liste; alle Operationen $\mathcal{O}(1)$

Deque (Doppelschlange):

- Operationen: `pushFront`, `pushBack`, `popFront`, `popBack`
- **Als Array:** zirkulärer Puffer mit `front/back`-Zeigern modulo Kapazität; $\mathcal{O}(1)$ amortisiert
- **Als Liste:** doppelt verkettete Liste; $\mathcal{O}(1)$

Beispiel 3.13 — Python-Entsprechungen

```
from collections import deque

# Stack (LIFO) via list
stack = []
stack.append(1)    # push
stack.pop()        # pop --  $\mathcal{O}(1)$  amortisiert

# Queue / Deque via collections.deque
q = deque()
q.append(1)         # pushBack --  $\mathcal{O}(1)$ 
q.appendleft(0)     # pushFront --  $\mathcal{O}(1)$ 
```

```
q.pop()           # popBack    --  $O(1)$   
q.popleft()       # popFront  --  $O(1)$ 
```

`list.pop(0)` ist **nicht** $O(1)$ — alle Elemente müssen verschoben werden! Daher `collections.deque` für Queue verwenden.

Prüfungsscheckliste — Kapitel 3: Grundlegende Container

- ☐ Adressierungsformel für Arrays aufschreiben: $\text{addr}(a[i]) = \text{addr}(a[0]) + i \cdot s$. → *Def. 3.1*
- ☐ Komplexitäten aller Array-Operationen kennen und begründen (wahlfreier Zugriff, Insert, Append naiv). → §1
- ☐ Verdoppelungsstrategie erklären: Wachstumsfaktor L , Schrumpffaktor B , warum muss $L < B$ gelten? → *Def. 3.2*
- ☐ Amortisierte $\mathcal{O}(1)$ für **append/pop** erklären — Satz 3.3 formulieren. → *Satz 3.3*
- ☐ Korrektheitsbedingung der amortisierten Analyse aufschreiben: $\sum T \leq \sum A$. → *Def. 3.4*
- ☐ Bankkonto-Methode anwenden: Kredite vergeben, zeigen dass Kontostand $\Phi \geq 0$ stets gilt. → *Def. 3.5*
- ☐ Binärzähler analysieren: Gesamtkosten $\leq 2m$, Geometrische Reihe ausrechnen. → *Bsp. 3.6*
- ☐ Dummy-Knoten erklären: warum vereinfacht er den Code? → *Def. 3.7*
- ☐ **splice**(a, b, t) definieren: Semantik, Vorbedingung, Komplexität $\mathcal{O}(1)$ begründen. → *Def. 3.8*
- ☐ Alle Listenoperationen auf **splice** zurückführen (insert, remove, moveAfter). → *Def. 3.8*
- ☐ Freelist erklären: Motivation, Funktionsweise, Komplexitätsvorteil. → *Def. 3.10*
- ☐ Einfach vs. doppelt verkettete Liste: Unterschied, warum kein **remove**(a) in $\mathcal{O}(1)$? → *Def. 3.11*
- ☐ Vergleichstabelle Array vs. Liste: alle Operationen korrekt eintragen, Trade-offs benennen. → §6
- ☐ Stack (LIFO), Queue (FIFO), Deque definieren: Operationen, Implementierungen, Komplexitäten. → *Def. 3.12*

- Warum ist `list.pop(0)` in Python *nicht* $\mathcal{O}(1)$? Welche Alternative → *Bsp. 3.13* verwenden?
-

Literatur: DMS §3.1, §3.2 · CLRS §10.2, §17.2